ATHLETA CENTRO / DOCUMENTO DE DISEÑO

Athleta Manager - Autonomous Software Delivery Pipeline con Human Product Gate

Un sistema de entrega de software autónomo, verificable y gobernado por decisiones humanas de producto.

Idea central

Codex trabaja de forma continua sobre tareas independientes, acumula resultados y solo interrumpe cuando aparece una decisión que cambia el producto, la seguridad o la integridad de los datos.

Versión 1.0 | 11 de septiembre de 2026 | Fuente funcional: Athleta Manager - Descripción de Sistema

Resumen ejecutivo

Este documento define el sistema de trabajo que usaremos para evolucionar Athleta Manager con agentes de IA sin perder control sobre el producto. La propuesta combina autonomía de ejecución con un punto humano claro: el Human Product Gate. Codex puede analizar, construir, revisar, probar y preparar un Draft PR; la persona responsable decide cuando existe ambigüedad funcional o cuando el cambio está listo para incorporarse.

Recomendación

Aprobar un piloto acotado de 3 tareas independientes. No construir todavía un supervisor sofisticado ni un “equipo” permanente de agentes: primero validamos el protocolo, la calidad y el costo con trabajo real.

Qué se propone

- Una cola de tareas pequeñas, con contexto y criterios de aceptación explícitos.
- Un flujo estándar: análisis funcional, implementación, review independiente, QA y Draft PR.
- Bloqueo aislado por tarea: una duda no frena las tareas independientes.
- Una bandeja de decisiones para que el usuario responda por lotes, cuando le resulte conveniente.
- Reglas de autoridad que impiden que la IA invente producto, exponga información sensible o despliegue sin autorización.

Qué no se propone

- Autonomía para decidir reglas de negocio, permisos, estados o cambios de alcance.
- Deploy automático a producción.
- Múltiples agentes conversando sin necesidad concreta.
- Versionado o auditoría general de cada cambio del sistema Athleta, que la definición funcional excluye.

Resultado buscado

Que el usuario pueda sentarse a revisar varias tareas y encontrar trabajo preparado para decidir, en lugar de tener que conducir cada paso cada diez minutos.

Consulta rápida

Si necesitás...	Consultá	Para qué sirve
Preparar una tarea	6. Definition of Ready	Confirmar que puede empezar sin adivinar producto.
Entender el recorrido	4. Flujo y estados	Saber qué ocurre en cada punto y cuál es la salida esperada.
Resolver una duda	10. Ambigüedades y decisiones	Distinguir decisión técnica, decisión de producto y bloqueo.
Aceptar una entrega	7. Definition of Done + 8. QA	Revisar evidencia antes del Human Product Gate.

1. Propósito y límites

El pipeline existe para acelerar la construcción correcta de Athleta Manager. No es un producto separado para administrar agentes; es un protocolo operativo que organiza cómo se analiza, implementa y valida cada cambio.

Fuente de verdad

La definición funcional vigente es **Athleta Manager - Descripción de Sistema.pdf**. Cuando el código, una documentación anterior o una sugerencia de un agente contradicen esa fuente, prevalece la definición funcional. Si la contradicción exige una decisión de producto, la tarea se detiene en DECISION_REQUIRED.

Principios rectores

- Simplicidad funcional: resolver un problema real con la menor cantidad razonable de pasos, estados y excepciones.
- Privacidad e integridad como límites: la simplificación no puede exponer historia clínica, escalar privilegios, destruir historia ni producir resultados económicos incoherentes.
- Cambios pequeños y verificables: cada tarea tiene un resultado acotado y criterios observables.
- Historia primero: los registros clínicos, autorías, responsabilidades y movimientos relevantes se preservan.
- La IA propone y ejecuta dentro de la autoridad asignada; las decisiones de producto quedan en manos humanas.
- No anticipar necesidades futuras con estados, automatizaciones o configuraciones sin uso actual.

Alcance de esta versión

Incluye	Queda fuera
Protocolo de tareas, roles, estados, gates, review, QA, seguridad de ramas, bandeja de decisiones, métricas y piloto.	Implementación de un supervisor autónomo permanente, integraciones externas, notificaciones automáticas o despliegue productivo sin gate humano.

2. Arquitectura conceptual

El pipeline se entiende como una línea de producción con controles. Cada tarea lleva consigo su contexto, su autoridad y su evidencia. La autonomía ocurre dentro de la tarea; la gobernanza decide cuándo puede avanzar entre gates.

La tarea avanza sola hasta completar el siguiente control.

Componentes conceptuales

Componente	Función	Regla de diseño
Cola de trabajo	Ordena tareas listas o desbloqueadas.	No espera a una tarea bloqueada si existe trabajo independiente.
Contexto de tarea	Conserva objetivo, archivos, criterios, riesgos y decisiones.	La tarea debe poder retomarse sin rehacer el análisis.
Agente ejecutor	Analiza, implementa y valida.	Opera solo dentro del nivel de autoridad asignado.
Review independiente	Busca errores, regresiones, deuda y desalineación funcional.	Debe tener mirada separada del agente que construyó.
Bandeja de decisiones	Agrupa preguntas que requieren intervención humana.	Interrumpe solo lo necesario y permite responder por lotes.
Human Product Gate	Autoriza cambios de producto y la incorporación del resultado.	No se reemplaza por una aprobación automática.

Regla de continuidad

Se bloquea la tarea, no el equipo

Si la tarea A necesita una decisión, A queda PARKED. Las tareas B, C y D continúan si no dependen de A. Solo un bloqueo global, excepcional y documentado, puede pausar más de una tarea.

3. Roles y funciones

Los roles se definen por función, no por una imitación rígida de una organización. Un mismo agente puede asumir más de una función en una tarea, pero la revisión crítica debe conservar independencia.

Rol	Responsabilidad	No puede decidir por sí solo
Usuario / Product Owner	Define problema, prioridad y decisiones de producto. Acepta o rechaza el resultado.	No necesita dirigir cada detalle técnico.
Agente de análisis	Contrasta la tarea con el PDF, detecta ambigüedades, propone alcance y criterios.	No inventa reglas, estados, permisos ni automatizaciones.
Agente builder	Implementa el alcance aprobado, preserva patrones existentes y agrega pruebas proporcionales.	No amplía alcance ni modifica reglas para acomodar el código.
Reviewer independiente	Revisa diff, seguridad, privacidad, integridad, UX y regresiones.	No redefine el producto sin elevar una decisión.
QA / verificador	Ejecuta criterios de aceptación, tests relevantes y checks de integración.	No “pasa” una tarea con fallas críticas conocidas.
Supervisor futuro	Observa estados, prioriza señales y detecta atascos.	No autoriza cambios de producto ni despliegues por defecto.

Separación mínima

En tareas de riesgo medio o alto, quien revisa no debe ser el mismo agente que produjo la solución. La independencia puede ser otro agente, otra ejecución o una revisión humana focalizada, siempre que evalúe evidencia fresca y el diff real.

4. Flujo operativo y estados

El flujo es deliberadamente corto. Cada estado representa una situación útil para decidir qué hacer a continuación, no una ceremonia.

Estado	Significado	Salida esperada
BACKLOG	Idea o trabajo todavía no preparado.	Tarea seleccionable o descartada.
READY	Objetivo, contexto y criterios suficientes para comenzar.	Análisis iniciado.
ANALYSIS	Se valida alcance, fuente funcional, riesgos y plan.	BUILD o DECISION_REQUIRED.
BUILD	Se implementa el cambio dentro del alcance.	REVIEW.
REVIEW	Se inspecciona de forma independiente.	QA, FIX_REQUIRED o DECISION_REQUIRED.
QA	Se verifican criterios y regresiones relevantes.	DRAFT_PR o FIX_REQUIRED.
DRAFT_PR	Resultado listo para revisión humana e incorporación.	HUMAN_PRODUCT_GATE.
PARKED	La tarea espera una decisión, dependencia o recurso.	Retoma desde el punto guardado.
DONE	Cambio aceptado e incorporado según el proceso vigente.	Métrica y aprendizaje registrados.
BLOCKED_GLOBAL	Continuar puede comprometer integridad, seguridad o consistencia de varias tareas.	Intervención prioritaria y diagnóstico.

Ciclo autónomo

ANALYSIS -> BUILD -> REVIEW -> QA puede ejecutarse sin mensajes intermedios. El sistema solo notifica por una decisión, un bloqueo relevante, una falla o la finalización de un Draft PR.

5. Niveles de autoridad

La autonomía se limita por impacto. Cuanto mayor sea el riesgo para producto, datos, privacidad o usuarios, más cerca está el gate humano.

Nivel	Puede hacer	Requiere gate humano
A - Mecánico seguro	Leer, analizar, escribir tests, corregir errores locales, formatear, documentar y preparar cambios acotados sin alterar contratos funcionales.	Solo si aparece una ambigüedad o un riesgo no previsto.
B - Cambio funcional acotado	Implementar una regla ya definida en el PDF y en la tarea, con pruebas y revisión.	Aceptar el Draft PR antes de incorporar.
C - Sensible / reversible	Proponer o implementar con protección adicional cambios de permisos, datos, migraciones, integraciones, privacidad o lógica económica.	Antes de aplicar migración, cambiar permisos, transformar datos o activar una integración.
D - Producto / irreversible	Analizar alternativas y preparar un plan o prototipo no destructivo.	Siempre: nueva regla, nuevo estado, nuevo rol, automatización, eliminación relevante, deploy productivo o cambio de alcance.

Reglas no negociables

- La IA nunca expone secretos, contraseñas, PIN, credenciales ni datos clínicos fuera del contexto autorizado.
- La IA no usa el ocultamiento visual como sustituto de la autorización backend.
- Una migración debe declarar compatibilidad, impacto, rollback o recuperación y datos afectados.
- No se eliminan registros históricos para simplificar una implementación.
- No se despliega a producción de forma autónoma en esta versión.

6. Definition of Ready

Una tarea está READY cuando el agente puede avanzar sin adivinar decisiones de producto. Si falta información que cambia el comportamiento, la tarea vuelve a BACKLOG o pasa a DECISION_REQUIRED.

Elemento	Mínimo requerido
Objetivo	Problema concreto que se quiere resolver y quién se beneficia.
Alcance	Qué incluye y qué queda expresamente fuera.
Fuente	Sección o regla relevante de la definición funcional vigente.
Criterios de aceptación	Comportamientos observables, incluyendo casos normales y excepciones relevantes.
Riesgo	Nivel A-D, datos afectados, permisos, privacidad, historia, dinero o migraciones.
Contexto técnico	Módulos, archivos o interfaces conocidas; no hace falta imponer una solución.
Decisión pendiente	Ninguna que altere producto, alcance, permisos, datos o UX crítica.

Formato de tarea recomendado

Título / objetivo / alcance incluido / fuera de alcance / fuente funcional / criterios de aceptación / riesgos / nivel de autoridad / dependencias / resultado esperado.

La tarea no debe ser “hacer Clínica” o “mejorar pagos”. Debe ser algo que pueda revisarse como una unidad, por ejemplo: “Permitir registrar un pago de Sala con importe distinto al Plan y exigir Motivo de ajuste”.

7. Definition of Done y review

La tarea está terminada cuando el resultado funciona, respeta la definición funcional y deja evidencia suficiente para que otra persona decida sin repetir todo el trabajo.

Definition of Done

- El cambio implementa exactamente el alcance aprobado.
- Los criterios de aceptación fueron verificados y su resultado quedó registrado.
- Los tests específicos y regresiones relacionadas pasan, o las excepciones están explicadas.
- Typecheck, lint y build se ejecutan cuando son relevantes al repositorio.
- La revisión independiente no deja hallazgos críticos ni altos abiertos.
- Permisos, privacidad, autorías, historia y consistencia económica fueron revisados cuando aplican.
- La experiencia responsive se verificó cuando el cambio es visible.
- El resultado está en Draft PR con resumen, evidencia, riesgos y decisiones pendientes.

Checklist de review independiente

Lente	Preguntas guía
Producto	¿Resuelve el problema? ¿Respeta el PDF? ¿Inventó estados o automatizaciones?
Datos	¿Preserva identidad única, historia, autoría y relaciones? ¿Hay migración o pérdida?
Seguridad	¿Backend aplica permisos? ¿Se limita la información clínica? ¿Se protegen secretos?
Calidad	¿Hay errores, casos borde, duplicación o regresiones previsibles?
UX	¿Es simple, sobrio, responsive y coherente con el lenguaje de Athleta?
Operación	¿Se puede probar, revertir o recuperar? ¿Queda claro qué debe hacer la persona?

8. QA y evidencia

QA no significa ejecutar todos los checks siempre. Significa elegir la validación suficiente para el riesgo real y hacer visibles sus resultados.

Tipo de tarea	Validación mínima	Evidencia
UI sin reglas sensibles	Criterios, tests afectados, responsive básico.	Capturas o recorrido, resultado de tests.
Regla de negocio	Casos normales, bordes y regresión del módulo.	Matriz de casos y resultados.
Permisos / privacidad	Pruebas positivas y negativas por rol; backend.	Matriz de acceso y pruebas.
Migración / datos	Esquema, compatibilidad, rollback/recuperación y datos representativos.	Plan de migración y verificación post-cambio.
Finanzas / historia clínica	Integridad, autoría, fechas, totales y no exposición de información.	Casos de integridad y revisión especializada.

Falla de QA

Una falla vuelve la tarea a FIX_REQUIRED con la evidencia del problema. El agente puede corregir y repetir la validación; si descubre una decisión de producto, pasa a DECISION_REQUIRED en vez de improvisar.

Checks que no pudieron ejecutarse se declaran expresamente. Una tarea no se marca DONE por compilar si todavía no se verificó su comportamiento relevante.

9. Git, branches, PRs y deploy

La seguridad operativa del pipeline depende de separar trabajo, revisión e incorporación. La rama y el Draft PR son límites de control, no burocracia.

Momento	Regla
Aislamiento	Cada tarea trabaja en una rama o worktree propio. No se mezclan cambios de tareas distintas.
Base	La tarea registra desde qué estado parte. Si la base cambia de forma relevante, se revalida antes de cerrar.
Commits	Pequeños, descriptivos y vinculados a la tarea. No incluir secretos, archivos sincronizados de referencia ni cambios ajenos.
Draft PR	Se crea cuando BUILD, REVIEW y QA tienen evidencia suficiente. Resume alcance, fuera de alcance, checks, riesgos y decisiones.
Merge	Solo después del Human Product Gate y de la verificación definida por el repositorio.
Deploy	No autónomo en el piloto. Requiere autorización explícita, checks críticos y plan de recuperación.
Rollback	Cada cambio sensible documenta cómo volver atrás o recuperar datos antes de aplicarse.

Aislamiento por tarea

Una tarea PARKED no debe dejar cambios compartidos que contaminen a otra. Si una dependencia técnica es común, se convierte en una tarea explícita o se coordina una base estable; no se resuelve mezclando trabajo silenciosamente.

10. Ambigüedades, bloqueos y decisiones

El criterio central es distinguir entre una duda que el agente puede resolver técnicamente y una duda que cambia el producto. Solo la segunda interrumpe al usuario.

Situación	Acción autónoma	Salida
Decisión técnica convencional	Elegir la alternativa más simple, reversible y consistente con el código existente.	Continuar y documentar.
Regla funcional definida	Implementar la regla y cubrir sus casos.	Continuar a review.
Ambigüedad de producto	Investigar contexto, presentar opciones y explicar impacto.	DECISION_REQUIRED / PARKED.
Riesgo de datos o privacidad	Detener la acción sensible, conservar diagnóstico y preparar alternativas seguras.	DECISION_REQUIRED o BLOCKED_GLOBAL.
Falla corregible	Corregir, repetir tests y volver a review.	FIX_REQUIRED -> REVIEW.
Dependencia externa	Avanzar todo lo que no dependa de ella y dejar la dependencia explícita.	PARKED parcial.

Bandeja de decisiones

Cada entrada debe ser breve y accionable: contexto, decisión exacta, opciones, recomendación, impacto de cada opción, tareas afectadas y qué puede seguir avanzando. El usuario puede responder varias entradas juntas. Una respuesta reanuda solo las tareas que dependen de esa decisión.

Bloqueo global: excepcional

Usarlo únicamente cuando seguir pueda corromper datos, mezclar ramas, escalar privilegios o dejar el repositorio en un estado inconsistente para varias tareas. Debe incluir causa, alcance afectado y plan de recuperación.

11. Cola de trabajo y autonomía

La cola permite que el usuario trabaje en paralelo sobre varias revisiones. La autonomía no se mide por cuánto tiempo el agente permanece activo, sino por cuántas tareas independientes entrega con evidencia confiable.

La tarea avanza sola hasta completar el siguiente control.

Política de scheduling

- Priorizar tareas READY de alto valor y bajo acoplamiento.
- No iniciar tareas que dependan de una decisión pendiente si existen alternativas independientes.
- Limitar el trabajo concurrente según capacidad de revisión humana; producir más Draft PRs de los que se pueden entender no es progreso.
- Reintentar automáticamente solo fallas transitorias y con límite; una falla determinista se eleva con evidencia.
- Agrupar preguntas de la misma decisión para reducir interrupciones.
- Cerrar o archivar tareas sin valor antes de convertirlas en deuda activa.

Ejemplo de bandeja al volver

Tarea	Estado	Qué encuentra el usuario
Pago con ajuste	DRAFT_PR	Implementación, tests y review listos.
Permiso de historia clínica	DECISION_REQUIRED	Dos alternativas y recomendación.
Agenda de Sala	QA	Criterios en verificación.
Migración de personas	PARKED	Dependencia explícita de una decisión de datos.

12. Piloto inicial de 3 tareas

El piloto debe validar el protocolo con cambios reales, acotados y de riesgo diverso. No busca probar todas las capacidades de IA ni construir infraestructura permanente.

Tarea piloto	Por qué elegirla	Riesgo / aprendizaje
A. Pago de Sala con importe distinto al Plan	Regla explícita y acotada; toca dinero, motivo de ajuste y periodos.	Validar integridad económica, fechas de pago y UX de excepción.
B. Filtro de Tratamientos sin Atención reciente	Regla operativa clara, sin decisión destructiva.	Validar consulta clínica, fecha de referencia y permisos.
C. Reasignación antes de inactivar Profesor	Caso de protección histórica y responsabilidad actual.	Validar bloqueo correcto, reasignación masiva y preservación de autorías.

Criterios de éxito del piloto

- Las 3 tareas llegan a un resultado revisable sin conducción paso a paso.
- Al menos una tarea atraviesa todo el flujo hasta Draft PR.
- Las preguntas que aparecen se agrupan y llegan como decisiones claras, no como interrupciones pequeñas.
- El review independiente detecta cualquier problema relevante antes del gate.
- El esfuerzo humano para revisar y decidir es razonable en relación con el valor entregado.
- No se introducen estados, permisos, automatizaciones o cambios de alcance no aprobados.

Salida del piloto

Retrospectiva de una página: qué funcionó, dónde se frenó, qué controles fueron útiles, qué costo tuvo y qué debe cambiar antes de ampliar la cola.

13. Métricas y aprendizaje

Las métricas deben servir para mejorar la entrega y la confianza, no para premiar actividad artificial. Medir menos cosas, pero con significado.

Métrica	Qué indica	Señal saludable
Lead time a Draft PR	Tiempo desde READY hasta resultado revisable.	Baja sin aumentar fallas o retrabajo.
Tiempo humano por tarea	Análisis, decisiones y revisión que consume el usuario.	Disminuye sin perder comprensión.
First-pass review	Tareas que pasan review sin hallazgo crítico/alto.	Sube con mejores tareas y contexto.
Retrabajo	Vueltas FIX_REQUIRED por tarea y causa.	Baja; las causas se convierten en aprendizaje.
Decisiones por tarea	Cuántas decisiones de producto aparecen.	Pocas y bien formuladas; no cero por ocultamiento.
Bloqueos globales	Eventos que comprometen más de una tarea.	Cero o excepcionales, con causa resuelta.
Cambios fuera de alcance	Desvíos detectados en review.	Cero cambios no aprobados.

Revisión periódica

Después de varias iteraciones relevantes, revisar módulos avanzados, gaps funcionales, decisiones abiertas, deuda técnica con riesgo real y alineación con el PDF. La próxima prioridad debe elegirse por valor para Athleta, no por facilidad para el agente.

14. Supervisor futuro y gobernanza

Un supervisor puede ser útil más adelante, pero solo después de validar el protocolo manualmente. Su función inicial debe ser observar y ordenar, no decidir producto.

Evolución por etapas

Etapa	Capacidad	Límite
1. Piloto	Cola, estados, evidencia y bandeja de decisiones.	Sin supervisor permanente.
2. Coordinación	Detectar tareas listas, dependencias y fallas repetidas.	No modifica alcance ni aprueba PRs.
3. Supervisión	Sugerir prioridad, agrupar decisiones y señalar riesgos.	Toda acción sensible requiere gate.
4. Automatización acotada	Reintentos seguros, checks rutinarios y housekeeping reversible.	Sin migraciones, permisos, merges o deploys autónomos por defecto.

Gobernanza

- La definición funcional sigue siendo la autoridad de producto.
- Las tareas sensibles requieren revisión explícita y evidencia.
- Las excepciones se registran con motivo, responsable y alcance.
- Los secretos y datos clínicos no forman parte del contexto salvo necesidad autorizada.
- Toda nueva automatización debe tener objetivo actual, límites, observabilidad y forma de desactivación.
- La velocidad nunca justifica saltar privacidad, integridad o revisión.

Decisión de expansión

Ampliar el nivel de autonomía solo si el piloto muestra calidad estable, bajo retrabajo, decisiones bien aisladas y revisión humana sostenible.

15. Riesgos y mitigaciones

Riesgo	Cómo aparece	Mitigación
Teatro de agentes	Muchos roles y documentos, poco valor real.	Mantener funciones mínimas; medir tiempo humano y calidad.
Alucinación de producto	Agente completa huecos inventando reglas.	Fuente funcional obligatoria; DECISION_REQUIRED ante ambigüedad.
Regresión silenciosa	Cambio local rompe permisos o historia.	Review independiente y pruebas por rol/datos.
Exceso de autonomía	Merge, migración o deploy sin comprensión humana.	Niveles de autoridad y Human Product Gate.
Contaminación entre tareas	Ramas, contexto o cambios mezclados.	Worktree/branch por tarea y base registrada.
Fatiga de revisión	El agente produce más de lo que se puede revisar.	Limitar concurrencia y priorizar tareas de alto valor.
Pérdida de contexto	La tarea se retoma sin decisiones ni evidencia.	Contexto persistente, resumen de handoff y criterios.

Conclusión

Athleta no necesita una fábrica de agentes compleja. Necesita una forma confiable de convertir decisiones de producto en incrementos terminados, sin que el usuario tenga que coordinar cada movimiento. El diseño recomendado es autónomo en la ejecución, prudente en los cambios sensibles y explícito en el momento exacto en que una decisión humana es necesaria.

Próximo paso

Ejecutar el piloto de las 3 tareas, registrar evidencia y hacer una retrospectiva antes de ampliar la autonomía o construir un supervisor.

Anexo: vocabulario operativo

Human Product Gate: punto donde una persona acepta, rechaza o redefine un resultado que puede cambiar el producto. **Draft PR:** entrega técnica revisable, todavía no incorporada. **Decision Required:** estado que indica una decisión de producto, seguridad, datos o alcance pendiente. **Parked:** tarea estacionada sin detener trabajo independiente.

Alineación funcional: toda la propuesta se apoya en la definición vigente de Athleta Manager: simplicidad, identidad única, roles combinables, privacidad clínica, integridad económica, preservación de historia y exclusiones explícitas.